

Punto 3.9 - Lazy Loading: configuración, funcionamiento y riesgos

Práctica

Ejercicio: Habilitar la carga Lazy en el proyecto AceriaData, marcar las propiedades de navegación como virtual, demostrar el problema N+1 y comparar el rendimiento con la carga Eager. Crear un caso de uso que ejecute las consultas con carga Lazy y analice el SQL generado.

Contexto del proyecto: En el punto 3.8 se profundizó en la carga Eager con Include, ThenInclude, Filtered Include y AsSplitQuery. En este punto se explora la carga Lazy con fines demostrativos, incluyendo su configuración, su funcionamiento y sus riesgos. Esta exploración servirá para justificar el uso de la carga Eager en los módulos posteriores.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Infrastructure
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Infrastructure → entra en la carpeta del proyecto de infraestructura.

Paso 2: Instalar el paquete de proxies

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies
```

dotnet add package → añade una referencia a un paquete NuGet. Microsoft.EntityFrameworkCore.Proxies → nombre del paquete que contiene los proxies para la carga Lazy.

Error común: si se olvida instalar el paquete, el método UseLazyLoadingProxies no está disponible y el código no compila.

Paso 3: Marcar las propiedades de navegación como virtual

Modificar las entidades del dominio en src/AceriaData.Domain/Entities/:

OrdenFabricacion.cs:

```
namespace AceriaData.Domain.Entities;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = string.Empty;
    public string? Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }

    public virtual List<PlanchaAcero> Planchas { get; set; } = new();
    public virtual DetalleOrden? Detalle { get; set; }
    public virtual CertificadoCalidad? Certificado { get; set; }
    public virtual List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

Línea 1: namespace AceriaData.Domain.Entities; → declara el espacio de nombres. Línea 3: public class OrdenFabricacion → declara la entidad. Línea 5: public int Id { get; set; } → clave primaria. Línea 6:

public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 7: public string Cliente { get; set; } = string.Empty; → cliente. Línea 8: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 9: public DateTime? FechaEntrega { get; set; } → fecha de entrega opcional. Línea 10: public string Estado { get; set; } = string.Empty; → estado. Línea 11: public string? Observaciones { get; set; } → observaciones opcionales. Línea 12: public bool IsDeleted { get; set; } → indica si está eliminada. Línea 13: public DateTime? DeletedAt { get; set; } → fecha de eliminación. Línea 15: public virtual List Planchas { get; set; } = new(); → colección de planchas. La palabra virtual permite que el proxy la sobrescriba. Línea 16: public virtual DetalleOrden? Detalle { get; set; } → detalle. La palabra virtual permite que el proxy la sobrescriba. Línea 17: public virtual CertificadoCalidad? Certificado { get; set; } → certificado. La palabra virtual permite que el proxy la sobrescriba. Línea 18: public virtual List OrdenesAleaciones { get; set; } = new(); → colección de entidades intermedias. La palabra virtual permite que el proxy la sobrescriba.

Error común: si las propiedades de navegación no son virtual, el proxy no puede sobrescribirlas y la carga Lazy no funciona. Se debe marcar todas las propiedades de navegación como virtual.

PlanchaAcero.cs:

```
namespace AceriaData.Domain.Entities;

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 15: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la referencia a la orden no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

Aleacion.cs:

```
namespace AceriaData.Domain.Entities;

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public decimal PorcentajeCarbono { get; set; }
    public decimal PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public bool Activo { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }

    public virtual List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

Línea 15: public virtual List OrdenesAleaciones { get; set; } = new(); → colección de entidades intermedias. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la colección no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

DetalleOrden.cs:

```
namespace AceriaData.Domain.Entities;

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 10: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la referencia no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

CertificadoCalidad.cs:

```
namespace AceriaData.Domain.Entities;

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 10: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba.

Error común: si la referencia no es virtual, el proxy no puede sobrescribirla y la carga Lazy no funciona.

OrdenAleacion.cs:

```
namespace AceriaData.Domain.Entities;

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; }
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = string.Empty;

    public virtual OrdenFabricacion Orden { get; set; } = null!;
    public virtual Aleacion Aleacion { get; set; } = null!;
}
```

Línea 10: public virtual OrdenFabricacion Orden { get; set; } = null!; → referencia a la orden. La palabra virtual permite que el proxy la sobrescriba. Línea 11: public virtual Aleacion Aleacion { get; set; } = null!; → referencia a la aleación. La palabra virtual permite que el proxy la sobrescriba.

Error común: si las referencias no son virtual, el proxy no puede sobrescribirlas y la carga Lazy no funciona.

| Paso 4: Configurar la carga Lazy en el DbContext

Modificar el método OnConfiguring del AceriaDbContext en src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs:

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder
            .UseSqlServer(
                "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;",
                sqlOptions =>
                {
                    sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                    sqlOptions.CommandTimeout(60);
                })
            .UseLazyLoadingProxies()
            .LogTo(
                Console.WriteLine,
                new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                LogLevel.Information)
            .EnableSensitiveDataLogging()
            .EnableDetailedErrors();
    }
}

```

Línea 1: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → método de configuración. Línea 3: `if (!optionsBuilder.IsConfigured)` → comprueba si las opciones ya están configuradas. Línea 5: `optionsBuilder` → objeto de configuración. Línea 6: `.UseSqlServer()` → registra el proveedor de SQL Server. Línea 7: `"Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"` → cadena de conexión. Línea 8: `sqlOptions =>` → delegado de configuración del proveedor. Línea 10: `sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);` → habilita los reintentos automáticos. Línea 11: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera. Línea 12: `}` → cierra el delegado. Línea 13: `.UseLazyLoadingProxies()` → habilita la carga Lazy con proxies. Línea 14: `.LogTo()` → habilita el logging. Línea 15: `Console.WriteLine,` → destino del logging. Línea 16: `new[] { "Microsoft.EntityFrameworkCore.Database.Command" },` → categorías. Línea 17: `LogLevel.Information)` → nivel mínimo. Línea 18: `.EnableSensitiveDataLogging()` → muestra los valores de los parámetros. Línea 19: `.EnableDetailedErrors();` → muestra información detallada en los errores.

Error común: si se olvida el `using Microsoft.EntityFrameworkCore;`, el método `UseLazyLoadingProxies` no está disponible y el código no compila.

| Paso 5: Crear el caso de uso de carga Lazy

Crear el archivo `src/AceriaData.Application/UseCases/CargaLazyUseCase.cs`:

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CargaLazyUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CargaLazyUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CARGA LAZY CON PROXIES ===");

        DemostrarCargaLazySimple();
        DemostrarProblemaN1();
        DemostrarCargaEagerComoAlternativa();
    }

    private void DemostrarCargaLazySimple()
    {
        Console.WriteLine("\n--- Carga Lazy simple ---");

        var orden = _unidad.Ordenes.ObtenerPorId(1);
        if (orden is not null)
        {
            Console.WriteLine($" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}");
            Console.WriteLine($" Tipo de la entidad: {orden.GetType().Name}");
            Console.WriteLine($" Planchas (carga Lazy): {orden.Planchas.Count}");
        }
    }

    private void DemostrarProblemaN1()
    {
        Console.WriteLine("\n--- Problema N+1 con carga Lazy ---");

        var ordenes = _unidad.Ordenes.ObtenerTodas();
        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");

        var totalPlanchas = 0;
        foreach (var orden in ordenes)
        {
            totalPlanchas += orden.Planchas.Count;
        }

        Console.WriteLine($"Total de planchas: {totalPlanchas}");
    }

    private void DemostrarCargaEagerComoAlternativa()
    {
        Console.WriteLine("\n--- Carga Eager como alternativa ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
        var totalPlanchas = ordenes.Sum(o => o.Planchas.Count);

        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Total de planchas: {totalPlanchas}");
    }
}

```

Línea 1: `using AceriaData.Application.Interfaces;` → importa las interfaces de la aplicación. Línea 3: `namespace AceriaData.Application.UseCases;` → declara el espacio de nombres. Línea 5: `public class CargaLazyUseCase` → declara el caso de uso. Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → campo de la unidad de trabajo. Línea 9: `public CargaLazyUseCase(IUnidadDeTrabajo unidad)` → constructor. Línea 11: `_unidad = unidad;` → asigna el parámetro al campo. Línea 14: `public void Ejecutar()` → declara el método principal. Línea 16: `Console.WriteLine("=== CARGA LAZY CON PROXIES ===");` → muestra la cabecera. Línea 18: `DemostrarCargaLazySimple();` → llama al método de carga

Lazy simple. Línea 19: `DemostrarProblemaN1()`; → llama al método del problema N+1. Línea 20: `DemostrarCargaEagerComoAlternativa()`; → llama al método de carga Eager como alternativa. Línea 23: `private void DemostrarCargaLazySimple()` → declara el método. Línea 25: `Console.WriteLine("\n--- Carga Lazy simple ---");` → muestra la cabecera. Línea 27: `var orden = _unidad.Ordenes.ObtenerPorId(1);` → carga la orden por Id. Línea 28: `if (orden is not null)` → comprueba si la orden existe. Línea 30: `Console.WriteLine($" Orden: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos. Línea 31: `Console.WriteLine($" Tipo de la entidad: {orden.GetType().Name}");` → muestra el tipo de la entidad, que es el proxy. Línea 32: `Console.WriteLine($" Planchas (carga Lazy): {orden.Planchas.Count}");` → accede a las planchas, lo que provoca la carga Lazy. Línea 36: `private void DemostrarProblemaN1()` → declara el método. Línea 38: `Console.WriteLine("\n--- Problema N+1 con carga Lazy ---");` → muestra la cabecera. Línea 40: `var ordenes = _unidad.Ordenes.ObtenerTodas();` → carga todas las órdenes. Línea 41: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes. Línea 43: `var totalPlanchas = 0;` → inicializa el contador. Línea 44: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 46: `totalPlanchas += orden.Planchas.Count;` → accede a las planchas, lo que provoca una consulta por cada orden. Línea 48: `Console.WriteLine($"Total de planchas: {totalPlanchas}");` → muestra el total. Línea 51: `private void DemostrarCargaEagerComoAlternativa()` → declara el método. Línea 53: `Console.WriteLine("\n--- Carga Eager como alternativa ---");` → muestra la cabecera. Línea 55: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();` → carga las órdenes con planchas usando `Include`. Línea 56: `var totalPlanchas = ordenes.Sum(o => o.Planchas.Count);` → suma las planchas sin consultas adicionales. Línea 58: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes. Línea 59: `Console.WriteLine($"Total de planchas: {totalPlanchas}");` → muestra el total.

Error común: si la carga Lazy no está habilitada, el acceso a `orden.Planchas` devuelve una colección vacía en lugar de cargar las planchas. Se debe asegurar que `UseLazyLoadingProxies` está configurado.

| Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CargaLazyUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 7: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CargaLazyUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito. Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso. Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 3, 10) };
    context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };

    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito. Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`. Línea 4: `context.Database.EnsureDeleted();` → elimina la base. Línea 5: `context.Database.EnsureCreated();` → crea la base. Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden. Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden. Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden. Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes. Línea 11: `context.SaveChanges();` → inserta las órdenes. Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha. Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha. Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha. Línea 16: `var plancha4 = new PlanchaAcero { ... };` → crea la cuarta plancha. Línea 18: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);` → registra las planchas. Línea 19: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 9: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tipo de la entidad (el proxy), el problema N+1 con las consultas adicionales y la carga Eager como alternativa.

Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CARGA LAZY CON PROXIES ===
```

```
--- Carga Lazy simple ---
```

```
Orden: OF-2024-0001 | Cliente: Constructora del Norte
Tipo de la entidad: OrdenFabricacionProxy
Planchas (carga Lazy): 2
```

```
--- Problema N+1 con carga Lazy ---
```

```
Órdenes cargadas: 3
... (3 consultas adicionales para cargar las planchas de cada orden)
Total de planchas: 4
```

```
--- Carga Eager como alternativa ---
```

```
Órdenes cargadas: 3
Total de planchas: 4
```

La primera sección muestra el tipo de la entidad, que es `OrdenFabricacionProxy`, confirmando que la carga Lazy está habilitada. La segunda sección muestra el problema N+1: se ejecutan tres consultas adicionales para cargar las planchas de cada orden. La tercera sección muestra la carga Eager como alternativa, que ejecuta una sola consulta.

Observaciones: en la primera sección, el acceso a `orden.Planchas.Count` provoca una consulta adicional. En la segunda sección, el bucle sobre las órdenes provoca una consulta por cada orden. En la tercera sección, la carga Eager carga las planchas en una sola consulta.

Paso 11: Diagnosticar un error común

Modificar el método `DemostrarProblemaN1` para acceder a la propiedad `Planchas` de cada orden dentro de un bucle anidado:

```
csharp
private void DemostrarProblemaN1()
{
    Console.WriteLine("\n--- Problema N+1 con carga Lazy ---");

    var ordenes = _unidad.Ordenes.ObtenerTodas();
    Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");

    foreach (var orden in ordenes)
    {
        Console.WriteLine($" Orden: {orden.NumeroOrden}");
        foreach (var plancha in orden.Planchas)
        {
            Console.WriteLine($"    Plancha: {plancha.Espesor} mm");
            Console.WriteLine($"    Aleaciones: {plancha.Orden.OrdenesAleaciones.Count}");
        }
    }
}
```

Resultado esperado: el bucle anidado accede a `plancha.Orden.OrdenesAleaciones`, lo que provoca consultas adicionales. Si hay tres órdenes y cuatro planchas, se ejecutan tres consultas para las órdenes, cuatro para las planchas y cuatro para las aleaciones de cada plancha. En total, once consultas.

Solución: usar `Include` con `ThenInclude` para cargar todas las entidades relacionadas en una sola consulta.


```
var ordenes = _unidad.Ordenes.ObtenerQueryable()
    .Include(o => o.Planchas)
    .ThenInclude(p => p.Orden)
    .ThenInclude(o => o.OrdenesAleaciones)
    .ToList();
```

Resultado esperado con la solución: una sola consulta carga todas las entidades relacionadas.

Errores comunes del ejercicio

Error	Causa	Solución
Carga Lazy no funciona	Las propiedades de navegación no son virtual	Marcar como virtual
Carga Lazy no funciona	No se configuró UseLazyLoadingProxies	Configurar en OnConfiguring
N+1 en bucle	Se accede a propiedades de navegación en un bucle	Usar Include
ObjectDisposedException	Se accede a propiedades de navegación fuera del ámbito	Usar Include o DTOs
Referencia circular en serialización	Las entidades se referencian mutuamente	Usar DTOs o configurar el serializador
NullReferenceException	La propiedad de navegación es null	Comprobar antes de acceder
Proxies no generados	Las clases son sealed	Quitar sealed

Reto resuelto: Comparar carga Lazy y carga Eager con medición de tiempo
 Reto: Crear un método en el caso de uso que mida el tiempo de ejecución de una consulta con carga Lazy y otra con carga Eager. Comparar los resultados y analizar la diferencia.

Solución paso a paso:

Paso 1: Añadir el método CompararRendimiento al caso de uso:

```
csharp
private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroLazy = System.Diagnostics.Stopwatch.StartNew();
    var ordenesLazy = _unidad.Ordenes.ObtenerTodas();
    var totalLazy = ordenesLazy.Sum(o => o.Planchas.Count);
    cronometroLazy.Stop();

    Console.WriteLine($"Carga Lazy: {cronometroLazy.ElapsedMilliseconds} ms | Total planchas: {totalLazy}");

    var cronometroEager = System.Diagnostics.Stopwatch.StartNew();
    var ordenesEager = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
    var totalEager = ordenesEager.Sum(o => o.Planchas.Count);
    cronometroEager.Stop();

    Console.WriteLine($"Carga Eager: {cronometroEager.ElapsedMilliseconds} ms | Total planchas: {totalEager}");
}
```

Línea 1: private void CompararRendimiento() → declara el método. Línea 3: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera. Línea 5: var cronometroLazy = System.Diagnostics.Stopwatch.StartNew(); → inicia el cronómetro para la carga Lazy. Línea 6: var ordenesLazy = _unidad.Ordenes.ObtenerTodas(); → carga todas las órdenes. Línea 7: var totalLazy = ordenesLazy.Sum(o => o.Planchas.Count); → suma las planchas con carga Lazy. Línea 8: cronometroLazy.Stop(); → detiene el cronómetro. Línea 10: Console.WriteLine(\$"Carga Lazy: {cronometroLazy.ElapsedMilliseconds} ms | Total planchas: {totalLazy}"); → muestra el tiempo de la carga Lazy. Línea 12: var cronometroEager = System.Diagnostics.Stopwatch.StartNew(); → inicia el cronómetro para la carga Eager. Línea 13: var ordenesEager = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude(); → carga las órdenes con planchas usando Include. Línea 14: var totalEager = ordenesEager.Sum(o => o.Planchas.Count); → suma las planchas con carga Eager. Línea 15: cronometroEager.Stop(); → detiene el cronómetro. Línea 17: Console.WriteLine(\$"Carga Eager: {cronometroEager.ElapsedMilliseconds} ms | Total planchas: {totalEager}"); → muestra el tiempo de la carga Eager.

Paso 2: Llamar al método desde Ejecutar:

```
CompararRendimiento();  
Paso 3: Ejecutar dotnet run y comparar los tiempos.
```

Resultado esperado: la carga Lazy tarda más que la carga Eager porque ejecuta más consultas. La diferencia es más notable cuando hay más órdenes.

Analogía final

La carga Lazy en una acería es como pedirle al archivo central que no te traiga las carpetas relacionadas hasta que las pidas. En lugar de recibir la carpeta de la orden con las planchas y el detalle adjuntos, recibes solo la carpeta de la orden. Si después necesitas las planchas, el archivo central te las trae en ese momento. Si necesitas el detalle, te lo trae después. Cada petición adicional es un viaje al archivo central. Si tienes cien órdenes y pides las planchas de cada una, haces cien viajes. Eso es el problema N+1. La carga Eager es como pedir todas las carpetas relacionadas de una vez: un solo viaje con todo lo necesario. La carga Lazy es cómoda porque no tienes que decidir de antemano qué necesitas, pero es ineficiente si haces muchas peticiones. En una acería con mucho volumen, la carga Eager es la opción correcta. En un prototipo o en una aplicación pequeña, la carga Lazy puede ser aceptable. Así funciona la carga Lazy en EF Core: cómoda pero peligrosa, útil en prototipos pero ineficiente en producción.

Resultado esperado

Al final del ejercicio, deberías haber:

Instalado el paquete Microsoft.EntityFrameworkCore.Proxies.

Marcado las propiedades de navegación como virtual.

Configurado UseLazyLoadingProxies en el DbContext.

Creado el caso de uso CargaLazyUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado la demostración de carga Lazy simple y verificado el tipo del proxy.

Observado el problema N+1 en el bucle.

Comprobado la carga Eager como alternativa.

Diagnosticado el error de acceder a propiedades de navegación en un bucle anidado.

Creado el método CompararRendimiento.

Conexión con el siguiente punto

En este punto se ha explorado la carga Lazy, incluyendo su configuración con proxies, su funcionamiento interno, el problema N+1 y sus riesgos en aplicaciones web y serialización. Se ha comprobado que la carga Lazy es cómoda pero ineficiente, y que la carga Eager es la alternativa recomendada. En el siguiente punto se estudiará la carga Explicit, que permite cargar entidades relacionadas bajo demanda de forma controlada.
